Formulación de problemas y espacios de búsqueda

Introducción

Hay un salto cualitativo importante entre un agente que reacciona a estímulos y uno que es capaz de planificar. Los primeros, las máquinas reactivas, tienen una relación directa entre entrada y salida: ante tal situación, tal respuesta. El problema es que ese diseño obliga a anticipar explícitamente todas las situaciones posibles que el agente puede enfrentar.

En muchos casos, eso es directamente impracticable. Y aun cuando fuese posible, el volumen de tablas y reglas que habría que predefinir y almacenar para tareas relativamente complejas demandaría grandes espacios de memoria y tiempos de consulta elevados. El desempeño del agente queda acotado y condicionado a los escenarios que alguien tuvo la previsión de contemplar al momento del diseño.

La alternativa es un agente que incorpore conductas adaptables, que no dependan de un diseño explícito anticipado. En lugar de almacenar respuestas predefinidas, este agente incorpora la capacidad de realizar procesos exploratorios para seleccionar las acciones más apropiadas en cada situación.

Russell y Norvig (2016) identifican a estos como agentes basados en objetivos (nivel 3 en su jerarquía de agentes) y, dentro de esa categoría, lo señalan como agentes que resuelven problemas. Lo que los caracteriza es que disponen de la capacidad analítica necesaria para anticipar los efectos de sus acciones, que les permite seleccionar conductas orientadas a alcanzar sus objetivos.

En este esquema, ya no existe una relación directa entre estímulo y respuesta. Por el contrario, la respuesta es el resultado de un proceso exploratorio de búsqueda cuya determinación constituye, en sí misma, la solución de un problema. Por ese motivo, la actividad de estos agentes queda encuadrada en un marco de planificación.

Ahora bien, la viabilidad de estos agentes depende en gran medida del entorno en el que operan. Como se analizó en el módulo anterior, existen distintos tipos de dominio: estáticos, dinámicos, interactivos y competitivos.

> Esta clasificación no es un detalle menor, ya que los algoritmos de búsqueda clásicos asumen, en su mayoría, que el entorno es estático, es decir, que solo cambia como consecuencia de las acciones del propio agente, y cuando ese supuesto se cumple, sus garantías teóricas aplican sin restricciones. Cuando el dominio es dinámico, interactivo o competitivo, esas garantías se debilitan o desaparecen, y el diseño del agente debe contemplar explícitamente esa complejidad adicional.

Caso: Google Maps y el problema de encontrar el camino óptimo

En 2023, Google Maps procesó más de mil millones de kilómetros de indicaciones de navegación al día, en más de 220 países y territorios. Detrás de cada mensaje del tipo 'gira a la derecha en 200 metros' existe un proceso de cómputo que resuelve, en milisegundos, un problema que en su versión más compleja involucra millones de nodos y aristas en un grafo de carreteras global.

Lo que hace a este caso especialmente instructivo no es la escala, sino el hecho de que el problema que Google Maps resuelve es exactamente el mismo tipo de problema que un robot autónomo resuelve para navegar por un almacén, que un sistema bioinformático resuelve para encontrar la ruta más corta en una red metabólica, o que un equipo de seguridad resuelve para trazar el camino de propagación de un malware a través de una red corporativa.

Google Maps modela el mundo como un grafo. Cada intersección es un nodo (un estado), cada tramo de calle es una arista (una acción disponible), y el costo de cada arista refleja tiempo estimado de viaje, distancia o una combinación ponderada de ambos. El estado inicial es la ubicación actual del usuario; el estado objetivo, su destino. Encontrar la ruta óptima es, en términos formales, un problema de búsqueda en este grafo.

Este caso ilustra los tres desafíos fundamentales que cualquier algoritmo de búsqueda debe enfrentar.

1. **Primero, la representación adecuada del problema:**
   - ¿Qué información debe contener cada estado?
   - ¿Qué constituye una acción válida?
2. **Segundo, la gestión de la complejidad:** el grafo de rutas global supera los 800 millones de nodos, lo que hace que un algoritmo ingenuo tarde horas en procesarlo.
3. **Tercero: la elección del algoritmo correcto:** ¿buscas el camino más corto en distancia, en tiempo o con tráfico en tiempo real? Cada criterio requiere una estrategia diferente.

Para resolver todo esto en milisegundos, Google Maps utiliza variantes avanzadas de los algoritmos que se estudiarán en esta lectura, combinadas con heurísticas geográficas. Sin embargo, la comprensión de los fundamentos es necesaria para entender por qué ciertas decisiones de diseño son correctas y otras no.

A modo de reflexión

Google Maps resuelve en milisegundos un problema de búsqueda sobre un grafo de más de 800 millones de nodos. Sin embargo, como la propia lectura señala, no lo hace con los algoritmos que estudiaremos aquí, sino que usa variantes avanzadas combinadas con heurísticas geográficas.

¿Qué crees que les falta a los algoritmos de búsqueda no informada para poder operar sobre un grafo de esa magnitud? ¿Qué información adicional necesitaría un algoritmo para no tener que explorar 800 millones de nodos antes de encontrar la ruta?

Anota tu respuesta. Volverás a esta pregunta al final de la lectura para ver cómo evolucionó tu perspectiva y se retomará con más profundidad en la próxima unidad, donde esa

información adicional tiene un nombre preciso.

Representación de problemas en inteligencia artificial

Antes de que una computadora pueda resolver un problema, es necesario representarlo y describirlo de manera adecuada. Sin embargo, esta tarea es más compleja de lo que parece.

Cuando una persona quiere ir de un lugar a otro, no piensa en pasos discretos ni en costos de transición. Integra lo que recuerda del camino, lo que percibe en ese momento, lo que intuye que va a pasar.

Una computadora no puede hacer eso, sino que necesita saber exactamente cuáles son las situaciones posibles, qué se puede hacer en cada una, qué pasa después de cada acción y cuándo considerar que el problema está resuelto. Esa traducción del problema real al computable es lo que se llama representación. De cómo se represente depende si el algoritmo funciona, si es eficiente o si directamente falla.

¿Qué es un problema?

Enfrentar un problema implica querer alcanzar cierto objetivo y conocer las acciones disponibles, pero no saber cuáles son las apropiadas ni en qué orden aplicarlas. Esa incertidumbre es precisamente lo que define su existencia y la decisión influirá directamente en la efectividad de la solución. Todo problema queda determinado por tres componentes básicos: una situación inicial, un objetivo a alcanzar y un conjunto de acciones posibles capaces de conducir a soluciones alternativas.

El objetivo puede ser algo concreto, como alcanzar una posición física en el espacio, o algo completamente abstracto, como demostrar un teorema matemático. En consecuencia, las acciones que llevan al objetivo serán, en algunos casos, operaciones físicas y, en otros, razonamientos o transformaciones puramente formales.

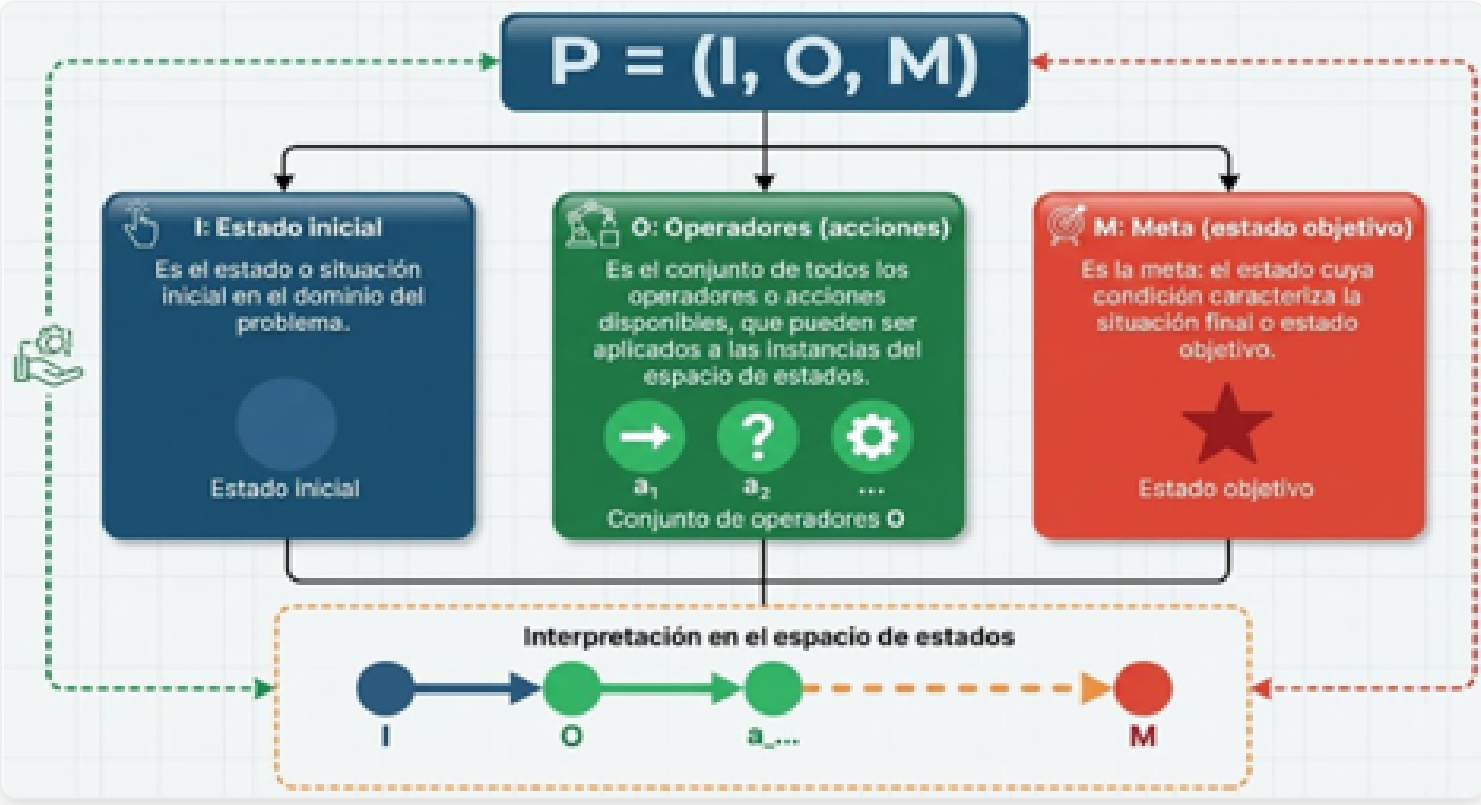
En inteligencia artificial, los problemas se representan como un espacio de estados, que constituye la base de la mayoría de los métodos de resolución. Esta representación tiene dos ventajas fundamentales: permite expresar formalmente el problema y posibilita definir el proceso de

resolución de manera sistemática. Bajo esta interpretación, un problema es la necesidad de convertir una situación inicial en una situación deseada, usando un conjunto de operaciones permitidas. Formalmente, un problema se define como una terna:

> *P = (I, O, M)*

Esta formalización fuerza a quien diseña el sistema a ser explícito sobre qué se sabe al inicio, qué está permitido hacer y qué significa haber terminado. Esta precisión es lo que distingue a un problema bien formulado de una descripción vaga de lo que se quiere lograr.

Figura 1. Representación formal de un problema



Fuente: elaboración propia.

El espacio de estados

El espacio de estados es el conjunto de todas las situaciones posibles en las que puede encontrarse el sistema. Cada estado representa una configuración específica del mundo en un momento dado.

Lo delicado acá es decidir qué información debe formar parte de esa configuración. Si se incluye demasiada información, el espacio de estados crece hasta volverse inmanejable. Si se incluye muy poca, el modelo queda incompleto y las soluciones obtenidas pueden no reflejar adecuadamente la realidad. La clave consiste en representar únicamente la información necesaria para decidir la siguiente acción, sin agregar detalles irrelevantes.

Algunos ejemplos ayudan a ver esto con claridad. Un robot que navega en un almacén necesita saber su posición y orientación, no la temperatura del ambiente. Un sistema que analiza la propagación de malware en una red necesita saber qué máquinas están comprometidas, no el historial completo de conexiones. Un algoritmo de alineamiento genético necesita saber en qué punto de cada secuencia está parado. En todos los casos, el estado captura lo suficiente para seguir adelante, sin cargar con información irrelevante.

Acciones y operadores

Las acciones son las operaciones disponibles para pasar de un estado a otro. También se las llama operadores, especialmente cuando se quiere enfatizar que funcionan como transformaciones: toman un estado y producen otro.

En navegación, las acciones son los movimientos posibles desde una intersección. En análisis de redes, una acción puede ser seguir una conexión TCP entre dos nodos. El concepto es siempre el mismo: una operación válida que lleva el sistema de una configuración a otra. Lo que cambia es el dominio.

Cada acción tiene una precondición: el estado tiene que permitir que esa acción tenga sentido y un efecto, el estado que resulta de aplicarla. Esta estructura es la base de los sistemas de planificación que se usan en robótica y en inteligencia artificial aplicada.

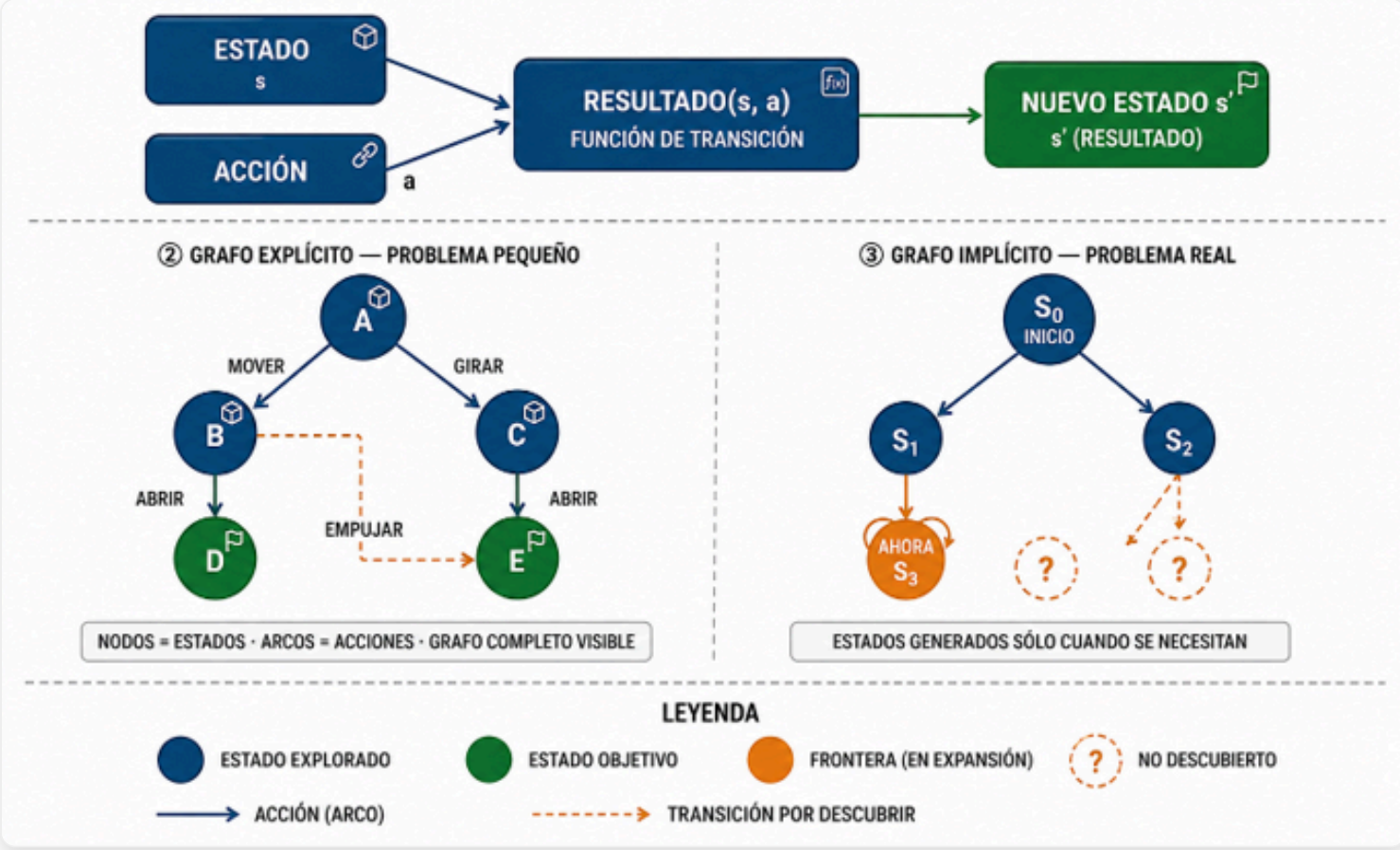
La función de transición

La función de transición describe qué ocurre cuando se aplica una acción sobre un estado determinado. Formalmente, suele representarse como:

> *Resultado (s, a) = s'*

Dado un estado s y una acción a, la función devuelve un nuevo estado s', que representa la situación resultante.

Figura 2. Función de transición y grafo del espacio de estados



Fuente: elaboración propia.

Gracias a esta función es posible conectar unos estados con otros. Si imaginas a cada estado como un nodo y a cada acción como un enlace entre nodos, obtendrás una estructura denominada grafo

del espacio de estados. En este grafo dirigido, los nodos representan estados y los arcos representan las acciones que permiten pasar de un estado a otro.

En problemas pequeños, este grafo puede construirse de manera explícita y visualizarse completo. Sin embargo, en la mayoría de los problemas reales, el número de estados es tan grande que resulta impracticable generarlo por adelantado. En esos casos, el grafo se considera implícito: los algoritmos van descubriendo y generando nuevos estados a medida que exploran el problema, construyendo únicamente las partes que necesitan analizar.

Estado inicial, objetivo y costo

El estado inicial es el punto de partida. El estado objetivo es la condición que se quiere alcanzar. El objetivo no siempre es un estado único, ya que muchas veces es una condición que varios estados pueden satisfacer. En navegación robótica, el objetivo puede ser cualquier punto dentro de una zona de carga. En seguridad informática, puede ser cualquier configuración de red donde la amenaza quedó contenida.

Finalmente, la función de costo de camino asigna un valor numérico a cada secuencia de acciones, lo que permite comparar soluciones entre sí. Cuando las acciones tienen costos diferentes, como un semáforo en rojo, un atasco, una reacción química más lenta, un enlace de red congestionada, el costo del camino suma los costos individuales de cada acción ejecutada. El objetivo del agente no es simplemente encontrar un camino hasta el objetivo, sino frecuentemente encontrar el de menor costo total. Korf (1985) demostró que, incluso en espacios de búsqueda enormes, diseñar bien la función de costo, en combinación con una buena heurística, puede reducir el esfuerzo computacional necesario para encontrar soluciones óptimas.

Figura 3: Componentes esenciales para representar un problema



Fuente: elaboración propia.

A modo de reflexión

Aplica los conceptos que acabas de aprender a un problema de tu carrera:

- **IA y robótica:** un robot de limpieza en un entorno de 5×5 habitaciones, ¿cuál es el estado? ¿Cuáles son las acciones? ¿Cuál es el objetivo?
- **Ciencia de datos:** un proceso de preprocesamiento de datos con 6 transformaciones posibles, ¿qué representa cada estado?
- **Seguridad informática:** un sistema que rastrea la propagación de un virus en una red de 20 nodos, ¿qué información debe contener cada estado?
- **Bioinformática:** un algoritmo que busca la ruta más corta entre dos genes en una red de interacciones, ¿qué es un estado? ¿Qué es una acción?

- **Innovación y desarrollo:** un sistema que sugiere características para un producto digital, ¿cómo se representa el espacio de posibilidades?

Para pensar: ¿Qué información incluiste en el estado? ¿Qué dejaste fuera? ¿Por qué?

Fundamentos de búsqueda en espacios de estados

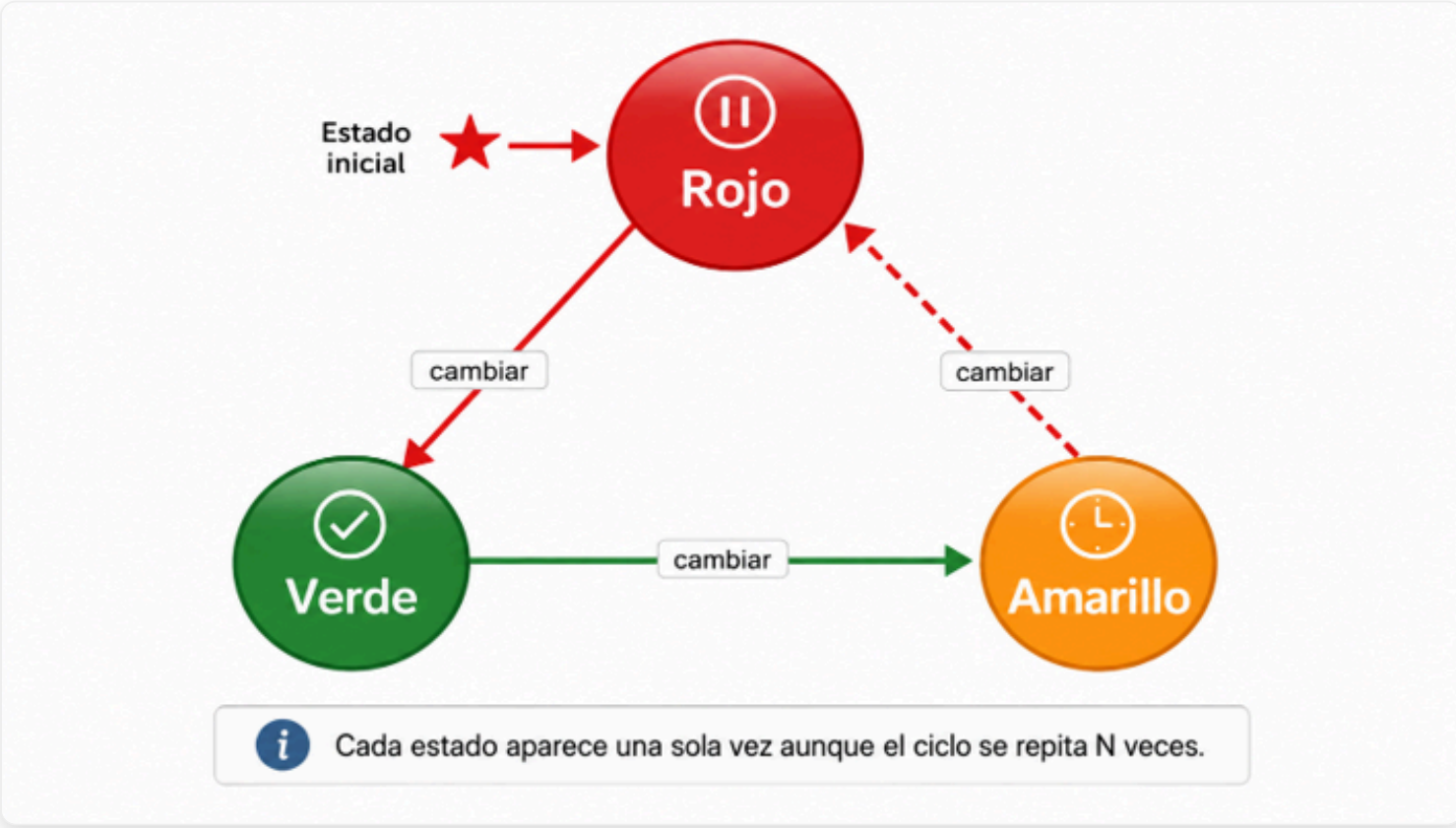
Antes de hablar de algoritmos, hay que tener claro con qué estructuras estás trabajando. Hay una confusión bastante común que vale la pena resolver desde el principio. El grafo del espacio de estados, el árbol de búsqueda y el árbol de soluciones no son lo mismo, aunque están relacionados. Mezclarlos genera problemas de análisis que después son difíciles de rastrear.

Grafo del espacio de estados

Piensa en el grafo del espacio de estados como el mapa del problema. Los nodos son estados y los arcos son acciones que llevan de un estado a otro. La característica central es que, si el mismo estado es alcanzable por diez caminos distintos, en el grafo aparece una sola vez, sin duplicados. Esa concisión es su gran ventaja porque representa el problema sin repetir información, pero tiene un costo.

El grafo puede contener ciclos. Un robot que gira 360 grados vuelve exactamente al estado del que partió. Un proceso de transformación de datos que aplica y luego revierte una operación queda donde mismo estaba. Un atacante que rebota entre dos máquinas en una red no está avanzando hacia ningún objetivo. Si el algoritmo no detecta esos ciclos, se queda girando indefinidamente sin llegar a ningún lado. Russell y Norvig (2016) dedican una sección entera a esto, ya que la detección y eliminación de ciclos es uno de los mecanismos de eficiencia más importantes en búsqueda.

Figura 4: Ejemplo de grafo



Fuente: elaboración propia.

En términos de implementación, el grafo puede representarse como lista de adyacencia o matriz de adyacencia. Pero en la práctica, la mayoría de los problemas reales trabajan con grafos implícitos y nunca se construye el grafo completo en memoria. Lo que se tiene es una función que, dado un estado, devuelve la lista de acciones disponibles y los estados a los que llevan. El grafo existe conceptualmente y se descubre a medida que el algoritmo avanza.

En el ejemplo anterior, si el semáforo pasa por el ciclo completo 100 veces, en el grafo siguen apareciendo solo 3 nodos, no 300. El ciclo Rojo→ Verde→ Amarillo→ Rojo es exactamente el tipo de ciclo problemático, ya que un algoritmo ingenuo podría recorrerlo infinitamente sin detectar que ya visitó esos estados.

Árbol de búsqueda

El árbol de búsqueda es lo que el algoritmo construye mientras trabaja (esta es la diferencia fundamental con el grafo). Si el mismo estado es alcanzable por cinco caminos distintos, en el árbol

aparece cinco veces, uno por cada camino. La raíz es siempre el estado inicial y cada nivel del árbol representa un paso más de profundidad desde ahí.

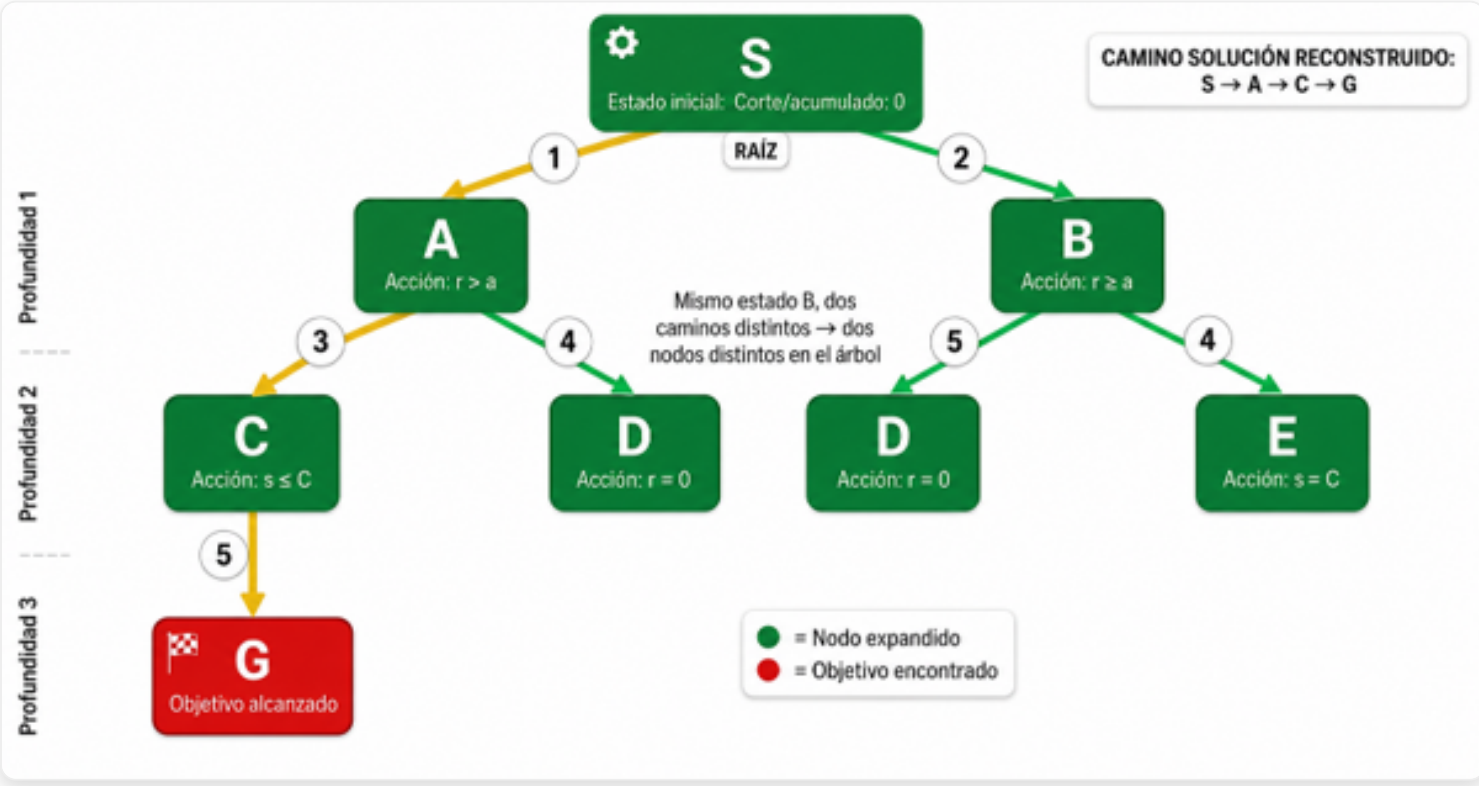
El árbol no solo registra qué estados se presentan, sino cómo se llega a cada uno. Cada nodo almacena el estado correspondiente, el nodo padre, la acción que lo generó y el costo acumulado desde la raíz. Sin esa información, cuando el algoritmo encuentra el estado objetivo, no puede reconstruir la secuencia de acciones que llevó hasta ahí, que es justamente lo que se necesita como solución.

> Un ejemplo concreto ayuda a entender la diferencia: imagina que buscas la ruta más corta entre dos nodos en una red de computadoras, puntualmente el camino que sigue un paquete desde un servidor de origen hasta un destino específico. El grafo del espacio de estados es esa red entera con todos sus routers y conexiones. El árbol de búsqueda que el algoritmo construye es la expansión progresiva de caminos posibles desde el nodo de origen, creciendo nivel por nivel hasta alcanzar el destino. Cada nodo del árbol recuerda el camino recorrido, lo que permite reconstruir la secuencia completa de saltos cuando se llega al destino. Sin esa memoria del recorrido, se llegaría al objetivo, pero sin entender cómo.

La distinción entre buscar en un grafo y buscar en un árbol tiene consecuencias prácticas que son importantes. La búsqueda en árbol es conceptualmente más simple, pero puede explorar el mismo estado múltiples veces, gastando tiempo y recursos en trabajo ya hecho. La búsqueda en grafo evita esa redundancia manteniendo una lista de estados ya explorados, lo que se llama lista cerrada, pero eso requiere memoria adicional para guardar todo lo visitado.

Para problemas pequeños, la diferencia es casi irrelevante. Para problemas del tamaño que aparecen habitualmente en ciencia de datos, seguridad informática o bioinformática, la elección puede ser la diferencia entre encontrar la solución en segundos o tardar horas, o entre usar megabytes o agotar la memoria disponible. Raschka y Mirjalili (2017) señalan que la gestión eficiente de memoria no es un detalle de implementación, sino una de las principales consideraciones prácticas cuando estos algoritmos se usan en sistemas reales.

Figura 5. Árboles de búsqueda



Fuente: elaboración propia.

Explosión combinatoria

Con esas estructuras claras, la pregunta que naturalmente aparece es cuánto puede crecer este espacio de búsqueda.

Uno de los resultados más contraintuitivos de las ciencias de la computación es que problemas que parecen simples a primera vista pueden generar espacios de búsqueda muy grandes. Este fenómeno se llama explosión combinatoria y es, probablemente, la razón más importante por la que la elección del algoritmo tiene consecuencias tan concretas en la práctica.

Para comprenderlo, es necesario considerar dos conceptos. El primero es el factor de ramificación (b), que representa el número promedio de acciones disponibles desde cada estado; es decir, cuántos nodos hijos se generan al expandir un nodo. El segundo es la profundidad (d), que indica cuántas acciones separan el estado inicial del objetivo y, por lo tanto, qué tan larga es la solución más corta.

El tamaño del árbol de búsqueda en el peor caso crece como b^d. Crecimiento exponencial puro. Con b = 10 y d = 10, el árbol tiene hasta 10 mil millones de nodos. Si d sube a 20, se habla de 100 quintillones. Ahora se presenta en el ajedrez, donde b ≈ 35 y d ≈ 80. El espacio resultante tiene más nodos que átomos en el universo observable. Russell y Norvig (2016) usan exactamente ese ejemplo para dejar en claro por qué los algoritmos de búsqueda no informada, por sí solos, no alcanzan para juegos complejos.

La explosión combinatoria no es solo un fenómeno de manual. En ciencia de datos, un pipeline de preprocesamiento con 20 transformaciones opcionales y 5 posibles parámetros cada una genera un espacio de 5^20, que son aproximadamente 95 mil millones de configuraciones. En seguridad informática, analizar de forma exhaustiva todos los caminos posibles de propagación de un ataque en una red de 500 máquinas es computacionalmente inviable sin heurísticas que guíen la búsqueda.

Para medir la eficiencia de un algoritmo de búsqueda de forma rigurosa, se utilizan dos métricas: el costo temporal mide cuántos nodos genera y procesa el algoritmo antes de encontrar la solución (o antes de concluir que no existe ninguna) y es proporcional al tiempo de ejecución. El costo espacial mide cuántos nodos necesita almacenar simultáneamente en memoria, que es una restricción igualmente crítica cuando los recursos son limitados.

La distinción entre los dos importa mucho según el contexto. Murphy (2022) señala que esta diferencia es especialmente relevante en aplicaciones modernas donde el cuello de botella puede ser cualquiera de los dos recursos. En sistemas embebidos como controladores de robots autónomos o dispositivos IoT, la memoria disponible puede ser tan ajustada que un algoritmo eficiente en tiempo pero hambriento de memoria resulta completamente inutilizable. En sistemas cloud de análisis de datos, en cambio, la memoria suele ser más elástica y el cuello de botella es el tiempo de cómputo. No hay una respuesta universal porque hay que entender el entorno donde va a correr el algoritmo.

Hay un tercer parámetro que aparece en algunos algoritmos. Es la profundidad máxima l, que es hasta donde el algoritmo está dispuesto a explorar. Si la solución está más profunda que l, no la va a encontrar, pero tampoco va a desperdiciar recursos recorriendo un árbol de tamaño ilimitado. Es un balance entre completitud y eficiencia que se verá en detalle más adelante.

Figura 6. Marco para comparación de algoritmos



Fuente: elaboración propia.

Antes de entrar a los algoritmos concretos, se necesita un marco común para compararlos. Russell y Norvig (2016) establecen cuatro propiedades estándar que permiten hablar de eficiencia de forma rigurosa y no solo intuitiva:

- **La primera es la completitud**, que responde si el algoritmo garantiza encontrar una solución cuando existe. Un algoritmo completo no se rinde antes de tiempo. Si hay un camino del estado inicial al objetivo, lo va a encontrar en tiempo finito. No todos los algoritmos cumplen esto y, en muchos contextos prácticos, la incompletitud es aceptable si a cambio se gana eficiencia, pero hay que saber cuándo se está haciendo ese sacrificio.

- **La segunda es la optimalidad**, que indica si la solución que devuelve es la de menor costo. Un algoritmo óptimo garantiza que no existe ninguna solución más barata. Esto es especialmente relevante cuando el costo de las acciones varía. En navegación, el camino más corto en distancia puede no ser el más rápido si los tramos tienen velocidades distintas. Encontrar una solución no es lo mismo que encontrar la mejor.

- **La tercera y la cuarta son el costo temporal y el costo espacial.** Se mencionan porque son parte del mismo marco de evaluación y porque en la práctica están frecuentemente en

> tensión. Los algoritmos que usan menos memoria tienden a generar más nodos, y viceversa. Entender ese balance es lo que permite elegir bien.

Algoritmos de búsqueda no informada

La búsqueda no informada, también llamada búsqueda a ciegas, es una familia de algoritmos que exploran el espacio de estados usando únicamente lo que la definición del problema les da: los estados, las acciones y la función de transición. No asumen nada sobre qué estados son más prometedores ni usan estimaciones externas de cuánto falta para llegar al objetivo. Eso los hace aplicables a cualquier problema bien definido, pero también los hace menos eficientes que los métodos informados que se verán más adelante.

Russell y Norvig (2016) organizan estos algoritmos según una pregunta simple: ¿qué nodo se expande primero? ¿El más cercano al inicio? ¿El más profundo? ¿El de menor costo acumulado? Cada respuesta genera un algoritmo distinto, con un perfil propio de completitud, optimalidad y eficiencia.

En el contexto de búsqueda en grafos, un nodo frontera o nodo límite es simplemente un nodo que ya fue generado, pero todavía no fue expandido. Cuando un algoritmo de búsqueda trabaja, va descubriendo nodos nuevos a medida que expande los que ya tiene. El momento en que un nodo aparece en el árbol, pero el algoritmo todavía no procesó sus hijos, ese nodo está en la frontera. Es como una lista de pendientes: ya se sabe que existe, ya se encontró, pero todavía no se vio qué hay más allá de él.

La frontera es exactamente la estructura de datos que diferencia a los algoritmos entre sí. BFS usa una cola FIFO, entonces el nodo que lleva más tiempo esperando en la frontera es el primero en expandirse, lo que produce la exploración nivel por nivel. DFS usa una pila LIFO, entonces el nodo más reciente es el primero en expandirse, lo que produce la exploración en profundidad. UCS usa una cola de prioridad ordenada por costo, entonces el nodo más barato es el primero en salir.

Un nodo deja de ser frontera en el momento en que el algoritmo lo expande, es decir, cuando genera todos sus sucesores y los agrega a esa misma frontera. En ese momento pasa a ser un nodo explorado o cerrado.

Entonces, para resumirlo, en cualquier momento de la ejecución de un algoritmo de búsqueda, el espacio se divide en tres zonas: los nodos ya explorados que el algoritmo procesó completamente, los nodos frontera que fueron descubiertos pero aún esperan ser procesados, y los nodos no descubiertos que el algoritmo todavía no sabe que existen.

Búsqueda en anchura (BFS)

BFS expande los nodos en orden de profundidad creciente. Primero, todos los estados alcanzables en un paso desde el inicio; luego, los alcanzables en dos pasos; luego, en tres, y así sucesivamente. Usa una cola FIFO como frontera, lo que garantiza que el primer nodo generado sea el primero en expandirse.

Un ejemplo concreto: imagina un sistema de análisis de redes en ciberseguridad que busca la ruta de propagación más corta de un malware desde un nodo infectado hasta un servidor crítico. BFS explora primero todos los vecinos directos del nodo infectado, luego los vecinos de esos vecinos, y así en adelante. La primera vez que llega al servidor crítico, encontró el camino más corto en número de saltos. No necesariamente el más rápido en tiempo de propagación, pero sí el de menor cantidad de pasos de red.

BFS es completo, es decir, que si existe una solución y el factor de ramificación b es finito, la va a encontrar. También es óptimo en un sentido específico, porque encuentra la solución más superficial, es decir, la de menor profundidad d. Pero eso solo equivale a la solución de menor costo si todas las acciones tienen el mismo costo unitario, condición que en problemas reales frecuentemente no se cumple.

El problema de BFS es la memoria. Necesita mantener en memoria todos los nodos del nivel actual antes de avanzar al siguiente, lo que en el peor caso significa almacenar O(b^d) nodos. Con b = 10 y d = 12, eso es aproximadamente un billón de nodos. Para cualquier problema de escala real, ese requerimiento hace a BFS impracticable si la solución está a más de unos pocos niveles de profundidad.

Dicho eso, BFS es ampliamente usado cuando la profundidad de la solución es pequeña o conocida de antemano. En análisis de redes sociales, por ejemplo, es la base del cálculo de grados de separación entre nodos.

Experimentación: visualización de BFS en acción

Explora cómo funciona BFS en un problema concreto de tu área.

- **Plataforma:** Registrar cuál IAG se utiliza.
- **Prompt sugerido:** tengo un grafo de 7 nodos donde el nodo A es el origen y el nodo G es el destino. Las conexiones son: A-B, A-C, B-D, B-E, C-F, C-G. Explícame, paso a paso, cómo funciona BFS para encontrar el camino más corto desde A hasta G. Muestra la cola en cada paso y los nodos ya visitados.
- **Luego pregunta:** ¿Cuántos nodos tuvo que explorar BFS antes de encontrar la solución? ¿Cuántos niveles de profundidad recorrió?

Reflexión: ¿Cambia el número de nodos explorados si el destino está en un nivel más profundo? ¿Qué implica eso para problemas más grandes de tu campo?

Búsqueda en profundidad (DFS)

DFS hace exactamente lo contrario que BFS. En lugar de explorar nivel por nivel, sigue un único camino hasta sus últimas consecuencias antes de retroceder. Usa una pila LIFO como frontera, lo que hace que siempre expanda el nodo más recientemente generado, profundizando tanto como sea posible en cada rama.

La metáfora más clara es la de un explorador en un laberinto que, al llegar a una bifurcación, siempre elige la primera opción y avanza hasta encontrar el objetivo o un callejón sin salida. Solo cuando no puede avanzar más, retrocede al último punto de bifurcación y prueba la siguiente opción.

DFS es especialmente útil cuando el espacio de búsqueda es muy profundo pero estrecho. En análisis de malware, por ejemplo, se usa para recorrer cadenas de llamadas en análisis estático: sigue el flujo de ejecución de una función hasta sus últimas instrucciones antes de volver al punto de origen.

Las propiedades de DFS son casi el complemento exacto de BFS. Su gran ventaja es el costo espacial: solo necesita almacenar el camino actual desde la raíz hasta el nodo en expansión, más los nodos hermanos no explorados en cada nivel. Eso da un costo espacial de $O(b \cdot m)$, donde m es la profundidad máxima del árbol, un ahorro enorme comparado con BFS.

Pero tiene dos debilidades importantes:

- **No es óptimo:** puede encontrar una solución subóptima si hay varias soluciones a distintas profundidades.
- **No es completo en espacios con ciclos o ramas infinitas:** puede quedarse explorando una rama indefinidamente sin encontrar jamás la solución. La solución a ese problema es la búsqueda limitada en profundidad.

Búsqueda de costo uniforme (UCS)

UCS es una generalización de BFS que considera el costo acumulado de los caminos en lugar de su longitud en pasos. En lugar de expandir el nodo más superficial, expande el de menor costo acumulado g(n) desde el inicio, usando una cola de prioridad ordenada por ese valor.

Cuando todos los costos son iguales, UCS y BFS son equivalentes. Cuando los costos varían, UCS es marcadamente superior. El ejemplo más claro es la navegación: ir de A a B puede ser muy corto en número de pasos, pero muy costoso en tiempo si el tramo pasa por una zona congestionada. UCS encuentra el camino de menor costo total, que es el verdaderamente óptimo según el criterio del problema.

En ciencia de datos, UCS se usa en búsqueda de hiperparámetros ponderada por costo computacional: si evaluar ciertos modelos es más costoso que otros, UCS prioriza explorar primero las configuraciones más baratas en tiempo de cómputo.

UCS es completo y óptimo, bajo una condición razonable, que el costo de cada acción sea mayor que cero ($\varepsilon > 0$). Esto es consecuencia directa de que UCS siempre expande primero el nodo de menor costo acumulado.

El costo temporal y espacial en el peor caso es $O(b^{(1 + C^*/\varepsilon)})$, donde C^* es el costo de la solución óptima y ε el costo mínimo de una acción. Dependiendo de los valores, eso puede crecer incluso más rápido que b^d.

Búsqueda limitada en profundidad (DLS)

DLS es DFS con una restricción explícita y es que el algoritmo no explora nodos más allá de una profundidad máxima l. Si al llegar a esa profundidad no encontró el objetivo, retrocede como en DFS normal. Los nodos en el límite se tratan como si no tuvieran sucesores.

Esa simple modificación resuelve el problema más grave de DFS, que son las ramas infinitas. Al imponer un límite, el algoritmo garantiza terminar en tiempo finito. Pero introduce un nuevo balance, si el límite l es menor que la profundidad real de la solución d, el algoritmo devuelve 'no encontrado' aunque la solución exista.

La elección de l depende del conocimiento del dominio. En planificación robótica, si se sabe que el robot necesita como máximo 15 pasos para llegar a cualquier punto del almacén, se utiliza l = 15. En análisis de propagación de malware, si la red tiene diámetro máximo de 8 saltos, l = 8 garantiza completitud. Cuando no tenemos ese conocimiento a priori, se necesita algo más sofisticado.

Visualiza los diferentes algoritmos de búsqueda

Accede a: https://clementmihailescu.github.io/Pathfinding-Visualizer/#

- Selecciona BFS y dibuja un laberinto simple. Observa cómo expande nodos nivel por nivel, como una ola.
- Cambia a DFS en el mismo laberinto. Observa cómo sigue un camino hasta el fondo antes de retroceder.
- **Observar:** ¿Cuál generó menos exploración innecesaria? ¿Cuál encontró el camino más corto? ¿Coincide con lo que predice la teoría?

Reflexión: ¿Cuál de estos patrones se parece más a cómo un malware se propaga en una red? ¿Cómo un robot navega en un almacén?

Comparación sistemática

No existe ningún algoritmo que sea simultáneamente óptimo, completo, eficiente en tiempo y eficiente en espacio para todos los casos. Esa ausencia de solución universalmente superior es una de las características más importantes de la búsqueda no informada, y uno de los principales motivadores para desarrollar métodos heurísticos. La siguiente tabla, basada en el análisis clásico de Russell y Norvig (2016), resume las propiedades de cada algoritmo:

Tabla 1. Comparación de los algoritmos

Criterio	BFS	DFS	Costo Uniforme	DLS (l < d)
Completitud	Sí (b finito)	No (ciclos/∞)	Sí (costos > 0)	No

Criterio	BFS	DFS	Costo Uniforme	DLS (l < d)
Optimalidad	Sí (costos = 1)	No	Sí (costos ≥ ε)	No
Costo temporal	O(b^d)	O(b^m)	O(b^1+C*/ε)	O(b^l)
Costo espacial	O(b^d)	O(b·m)	O(b^1+C*/ε)	O(b·l)
Ventaja clave	Encuentra la solución más corta.	Muy poco uso de memoria.	Garantiza un costo mínimo.	Memoria controlada.
Desventaja clave	Memoria exponencial.	No completo, no óptimo.	Lento, sin costos bajos y espacio denso.	Incompleto si l < d.

Fuente: elaboración propia.

Aplicaciones prácticas de la búsqueda en espacios de estados

Hasta acá se construyó el marco conceptual. Supiste qué es un espacio de estados, cómo representar un problema formalmente y cómo funcionan los principales algoritmos de búsqueda no informada. Ahora queda ver cómo esos mismos conceptos, con distintos nombres e implementaciones, aparecen en distintos dominios.

Navegación robótica

Imagina un robot de limpieza en un depósito rectangular. El depósito es una grilla de celdas, el robot empieza en una esquina y tiene que llegar a la estación de carga ubicada en otra celda. Las acciones posibles son moverse arriba, abajo, izquierda o derecha. Algunas celdas están bloqueadas por estanterías. Este es uno de los problemas más simples de planificación robótica, y los cuatro algoritmos se comportan de forma distinta sobre él.

BFS explora celda por celda en capas concéntricas desde el origen. Garantiza encontrar el camino con menos movimientos. Si la estación de carga está a 7 pasos, BFS la encuentra sin explorar caminos de 8 o más.

DFS elige una dirección y la sigue hasta chocar con una pared o salir del mapa, luego retrocede. Puede encontrar la estación de carga dando un rodeo innecesario, pero usa mucha menos memoria que BFS porque no necesita guardar toda la frontera.

UCS entra en juego cuando el robot consume más energía en ciertas celdas, por ejemplo, las que tienen alfombra o superficies irregulares. El costo de moverse no es uniforme y UCS garantiza encontrar el camino de menor gasto energético total, aunque no sea el de menos pasos.

DLS es útil cuando el robot tiene batería limitada: si puede dar como máximo 10 pasos antes de necesitar recargarse, DLS busca la estación de carga con ese límite de profundidad. Si no la encuentra en 10 pasos, concluye que no es alcanzable con la carga actual.

Análisis de una red de usuarios

Imagina una red social pequeña representada como un grafo. Cada nodo es un usuario y cada arista indica que dos usuarios son amigos. Quieres responder distintas preguntas sobre esa red.

- **BFS responde:** ¿cuántos grados de separación hay entre el usuario A y el usuario B? Explora los amigos directos de A, luego los amigos de esos amigos y así. Cuando llega a B, sabe que encontró el camino más corto en cantidad de conexiones. Esta misma lógica se usa para calcular métricas de cercanía en redes reales.

- **DFS responde:** ¿están A y B conectados de alguna forma, aunque sea indirecta? No importa cuántos pasos ni cuál es el camino, solo si existe alguno. DFS sigue conexiones en profundidad y devuelve sí o no. Es la base del cálculo de componentes conectadas: grupos de nodos que se pueden alcanzar entre sí.

- **UCS entra cuando las aristas tienen peso**, por ejemplo, la frecuencia de interacción entre usuarios. En ese caso, la pregunta cambia: ¿cuál es el camino de mayor afinidad acumulada entre A y B? UCS elige en cada paso la arista de mayor peso, garantizando la ruta óptima según ese criterio.

- **DLS es útil para análisis acotados:** ¿puede A influenciar a B en a lo sumo 3 pasos? En estudios de difusión de información o campañas de marketing, limitar la profundidad de búsqueda hace el análisis computacionalmente viable en redes con millones de nodos.

Seguridad informática

Imagina una red corporativa modelada como un grafo. Cada nodo es una máquina y cada arista representa una conexión posible entre ellas. Un atacante comprometió la máquina A y quiere llegar al servidor de base de datos B sin ser detectado.

- **BFS responde:** ¿cuál es el camino con menos saltos entre A y B? Menos saltos generalmente implica menos tiempo de exposición y menos oportunidades de detección. Los equipos de seguridad usan este análisis, llamado análisis de alcanzabilidad, para identificar qué tan expuesto está un activo crítico desde cualquier punto de la red.

- **DFS responde:** ¿existe algún camino posible entre A y B? En auditorías de segmentación de red, la pregunta no es cuál es el mejor camino sino si hay alguno. DFS recorre la red en profundidad y confirma o descarta la conectividad.

- **UCS agrega realismo:** cada conexión tiene un costo que representa la dificultad de explotarla, por ejemplo, el nivel de parche del sistema o la complejidad de la vulnerabilidad. UCS encuentra el camino de menor resistencia total: la ruta que un atacante con recursos limitados elegiría para llegar a B con el menor esfuerzo.

- **DLS aparece en el análisis de malware.** Cuando un sistema analiza el grafo de llamadas de un programa sospechoso buscando patrones maliciosos, limita la profundidad de búsqueda para evitar quedar atrapado en recursiones infinitas. Si la cadena de llamadas maliciosa no aparece en los primeros N niveles, se descarta esa rama.

Bioinformática

Una red metabólica es un grafo donde los nodos son compuestos químicos y las aristas son reacciones enzimáticas que transforman un compuesto en otro. La pregunta central es: dado un compuesto de partida, ¿cómo se puede sintetizar un compuesto objetivo?

- **BFS responde cuál es la ruta con menos reacciones enzimáticas entre dos compuestos.** Si el objetivo es encontrar la síntesis más directa posible, sin considerar el costo energético de cada paso, BFS garantiza esa solución.

- **DFS se usa para exploración:** ¿existe alguna secuencia de reacciones que conecte el compuesto A con el compuesto B? En redes metabólicas grandes, a veces la pregunta es simplemente si la ruta existe, sin importar cuántos pasos tiene.

- **UCS incorpora el costo energético de cada reacción.** No todas las reacciones son igual de costosas para la célula: algunas consumen más ATP, otras dependen de condiciones específicas. UCS encuentra la ruta de menor costo energético total, que es la biológicamente más probable o eficiente.

- **DLS es útil cuando se quiere buscar rutas de síntesis realistas.** Una ruta metabólica con 20 pasos es biológicamente improbable. Limitar la búsqueda a rutas de, por ejemplo, 6 reacciones como máximo reduce el espacio de búsqueda y devuelve solo rutas que tienen sentido en la práctica.

Planificación del desarrollo de un producto

Un equipo de desarrollo tiene un conjunto de funcionalidades para construir. Algunas dependen de otras: no se puede implementar el sistema de pagos si antes no existe el sistema de usuarios. Este grafo de dependencias es el espacio de estados, y el objetivo es encontrar un orden de desarrollo que sea válido y eficiente.

- **BFS responde:** ¿cuál es la secuencia de menor cantidad de etapas para llegar a tener la funcionalidad X lista? Explorando nivel por nivel desde las funcionalidades base, identifica el camino más corto en pasos de desarrollo hasta el objetivo.

- **DFS se usa para verificar si el plan es posible:** ¿existe algún orden válido de implementación que no genere dependencias circulares? Si DFS encuentra un ciclo en el grafo de dependencias, el plan tiene un error lógico: dos funcionalidades que se necesitan mutuamente y ninguna puede construirse primero.

- **UCS agrega costos reales al análisis.** Cada funcionalidad tiene un costo de desarrollo en tiempo o dinero, y algunas rutas hacia el producto final son más baratas que otras. UCS encuentra la secuencia de menor costo total, lo que permite priorizar el trabajo según el presupuesto disponible.

- **DLS aparece en la planificación por sprints.** Si un equipo trabaja en ciclos de dos semanas, puede explorar qué funcionalidades son alcanzables en a lo sumo N pasos de desarrollo dentro del sprint. Limitar la profundidad convierte un problema de planificación abierto en uno acotado y ejecutable.

Aplicación al caso

Los conceptos desarrollados a lo largo de esta lectura permiten responder a la pregunta que el caso dejó abierta: ¿por qué los algoritmos de búsqueda no informada son insuficientes para resolver el problema de Google Maps a escala real? Cada limitación identificada en algoritmos

como BFS, DFS, UCS y DLS tiene una correspondencia directa con las decisiones de diseño que un sistema de esa magnitud debe tomar.

Es importante empezar por lo más evidente. BFS no funciona en un grafo del tamaño de una red vial real y no es por algún detalle de implementación que se pueda corregir, el problema es matemático. Para garantizar que encuentra el camino más corto, el algoritmo necesita tener en memoria todos los nodos de cada nivel antes de avanzar al siguiente. Cuando el grafo tiene cientos de millones de intersecciones y cada una conecta con varias más, esa frontera crece de forma exponencial hasta superar cualquier cantidad de memoria disponible. No hay forma de hacer que eso escale.

UCS resuelve una cosa distinta y la resuelve bien. Siempre expande primero el nodo con menor costo acumulado desde el origen, lo que garantiza que la solución encontrada es óptima. El inconveniente es que no sabe nada sobre dónde está el destino. Sin esa información, el algoritmo explora en todas las direcciones por igual, y en un grafo de ochocientos millones de nodos puede gastar millones de expansiones alejándose del objetivo sin saberlo. La optimalidad no sirve de mucho si el proceso tarda días.

Hay algo que muchas veces se pasa por alto cuando se estudia esto y que el caso de Google Maps deja muy claro. Antes de elegir un algoritmo, hay que decidir qué representa un estado. Maps no guarda en cada nodo el historial de calles recorridas, ni el combustible restante, ni ningún otro dato contextual. Solo almacena la intersección actual y el costo acumulado para llegar hasta ahí. Esa decisión mantiene el espacio de búsqueda en un tamaño manejable. Agregar información irrelevante al estado no mejora la solución; solo multiplica los estados posibles sin ningún beneficio.

El otro punto que no es negociable en grafos reales es la detección de ciclos. Una red vial tiene bucles por todas partes, y sin un registro de los nodos ya visitados, cualquier algoritmo puede quedar dando vueltas indefinidamente por las mismas intersecciones. Mantener ese registro tiene un costo en memoria, pero en este contexto no es opcional.

Todo esto no significa que los algoritmos no informados estén mal diseñados. Significa que fueron concebidos para otro rango de problemas. Cuando el espacio de búsqueda es enorme y no se dispone de ninguna pista sobre dónde está el objetivo, la exploración ciega simplemente no alcanza. Y esa limitación es exactamente lo que motiva los métodos heurísticos: incorporar una

estimación de cuánto falta para llegar, sin garantías perfectas, pero con una capacidad para orientar la búsqueda que cambia completamente qué tipo de problemas se vuelven abordables.

Actividades de repaso de lecturas

ACTIVIDADES DE REPASO Y REFUERZO

PREGUNTAS VERDADERO/FALSO

Un agente basado en objetivos puede operar con las mismas garantías en cualquier tipo de entorno, ya que su capacidad de planificación le permite anticipar los efectos de sus acciones independientemente de si el dominio es estático, dinámico o competitivo.

Es verdadero, porque la planificación le otorga al agente la capacidad de anticipar cualquier cambio externo y seleccionar la acción más adecuada sin importar el tipo de entorno.

Es falso, porque la capacidad de planificación del agente supone que el entorno solo cambia como consecuencia de sus propias acciones. Cuando esa condición no se cumple, las garantías del proceso de búsqueda se debilitan o desaparecen.

Justificación:

Es falso. Un agente basado en objetivos planifica asumiendo que puede anticipar los efectos de sus acciones, supuesto que solo se sostiene en entornos estáticos. En dominios dinámicos, interactivos o competitivos, el entorno puede cambiar por causas ajenas al agente, lo que invalida los planes trazados y exige que el diseño contemple explícitamente esa complejidad adicional.

Referencias

Korf, R. E. (1985). Depth-first iterative-deepening: An optimal admissible tree search. Artificial Intelligence, 27(1), 97–109. https://doi.org/10.1016/0004-3702(85)90084-0

Murphy, K. P. (2022). Probabilistic machine learning: An introduction. MIT Press.

Raschka, S., Mirjalili, V. (2017). Python machine learning: Machine learning and deep learning with Python, scikit-learn, and TensorFlow (2nd ed.). Packt Publishing.

Russell, S., Norvig, P. (2016). Artificial intelligence: A modern approach (3rd ed.). Pearson.